

Table des matières

Labo 08 OLTP — Données semi-structurées sur le transactionnel.....	1
Le terrain, en une minute	1
Partie 1 — Fabriquer le document.....	2
Partie 2 — Interroger, typer, aplatir	3
Partie 3 — L’index GIN, ce que l’entrepôt n’a pas	6
Partie 4 — La comparaison qui conclut le module	7
Partie 5 — Ce que le contraste vous a montré	8
Nettoyage.....	9
Et ensuite ?.....	9

Labo 08 OLTP — Données semi-structurées sur le transactionnel

Laboratoire du Cours 08, voie OLTP · PostgreSQL + AdventureWorks OLTP · Formation SQL avancé

On refait le laboratoire OLAP, étape pour étape, sur la base transactionnelle Adventureworks dans PostgreSQL. Connectez-vous avec pgAdmin, DBeaver ou psql. Comptez trois quarts d’heure.

Les deux voies — Ce labo est le jumeau de Labo_08_OLAP.md. Les deux moteurs partagent les flèches -> et ->>, et presque tous les pièges, et un sujet n’existe que de ce côté : l’index GIN, qu’on va créer et mesurer. Gardez les deux fichiers ouverts côte à côte.

Pourquoi on fabrique la donnée — Adventureworks ne contient aucun JSON, et la formation refuse d’ajouter un téléchargement de plus. En construisant le document vous-même, vous comprenez sa forme avant de l’interroger.

Le terrain, en une minute

Deux tables suffisent : sales.salesorderheader pour la commande, sales.salesorderdetail pour ses lignes. On travaillera dans un schéma dédié, le bac à sable du Cours 03, section 1, facile à supprimer à la fin.

```
CREATE SCHEMA IF NOT EXISTS labo08;  
SET search_path TO labo08, public;
```

Partie 1 — Fabriquer le document

1.1 Un premier objet

Théorie : Theorie_08.md, section 3.

```
SELECT jsonb_build_object('produit', d.productid,  
                          'qte',    d.orderqty,  
                          'prix',   d.unitprice) AS ligne  
FROM   sales.salesorderdetail AS d  
LIMIT 3;
```

Attendu : trois documents à trois clés, chacun sur une ligne ; jsonb_pretty(...) les met en forme si vous voulez les lire confortablement.

1.2 Un document imbriqué

Théorie : Theorie_08.md, section 3.

```
CREATE TABLE labo08.commande AS  
SELECT h.salesorderid AS numero,  
       jsonb_build_object(  
         'commande', h.salesorderid,  
         'client',   h.customerid,  
         'date',     h.orderdate,  
         'lignes',   (SELECT jsonb_agg(jsonb_build_object('produit', d.productid,  
                                                           'qte',    d.orderqty,  
                                                           'prix',   d.unitprice))  
                     FROM   sales.salesorderdetail AS d  
                     WHERE  d.salesorderid = h.salesorderid)  
       ) AS document  
FROM   sales.salesorderheader AS h;  
  
SELECT COUNT(*) FROM labo08.commande;  
SELECT jsonb_pretty(document) FROM labo08.commande LIMIT 1;
```

Attendu : plus de trente et un mille commandes. La sous-requête est corrélée, c'est le geste du Cours 07 réemployé pour imbriquer les lignes dans leur commande. Remarquez au passage que l'entrepôt, lui, obtenait le même résultat avec un simple GROUP BY : le schéma normalisé oblige ici à descendre chercher les lignes dans une autre table.

1.3 Le piège de la clé nulle, à l'envers

Théorie : Theorie_08.md, section 3.

```
SELECT jsonb_build_object('a', 1, 'b', NULL) AS avec_null;
```

Attendu : les **deux** clés sont présentes, b valant null. DuckDB fait de même dans le labo jumeau, et les deux terrains de la formation s'accordent donc. Ce n'est pas vrai partout : d'après sa documentation, Snowflake écarte la clé. Un document fabriqué sur deux moteurs à partir de la même donnée n'a donc pas toujours les mêmes clés, et un test de présence de clé ne donne pas toujours le même résultat.

1.4 json contre jsonb, en une expérience

Théorie : Theorie_08.md, section 2.

```
SELECT '{"b":1, "a":2, "a":3}':::json AS type_json,  
       '{"b":1, "a":2, "a":3}':::jsonb AS type_jsonb;
```

Attendu : le premier rend le texte tel quel, doublon compris et ordre conservé. Le second dédoublonne (la dernière valeur gagne), réordonne les clés et normalise les espaces. C'est toute la différence entre les deux types, en une ligne.

Partie 2 — Interroger, typer, aplatis

2.1 Les quatre opérateurs

Théorie : Theorie_08.md, section 4.

```
SELECT document -> 'client'           AS fleche_simple,  
       document ->> 'client'          AS fleche_double,  
       document #> '{lignes,0,produit}' AS chemin_jsonb,  
       document #>> '{lignes,0,produit}' AS chemin_texte  
FROM labo08.commande  
LIMIT 5;
```

Regardez les types renvoyés. Les colonnes en flèche simple gardent la forme d'un document ; les autres rendent du texte nu. La règle : flèche simple tant qu'on descend, flèche double au dernier cran.

2.2 Le tri alphabétique déguisé

Théorie : Theorie_08.md, section 5.

```
-- Sans conversion : tri ALPHABÉTIQUE.  
SELECT document ->> 'commande' AS numero,  
       document #>> '{lignes,0,prix}' AS premier_prix  
FROM labo08.commande  
ORDER BY document #>> '{lignes,0,prix}' DESC  
LIMIT 10;
```

```
-- Avec conversion : tri numérique.
SELECT document ->> 'commande' AS numero,
      (document #>> '{lignes,0,prix}')::numeric AS premier_prix
FROM   labo08.commande
ORDER BY (document #>> '{lignes,0,prix}')::numeric DESC
LIMIT 10;
```

Comparez les deux listes. La première place 99 avant 1000, parce que le caractère 9 vient après le caractère 1. Le résultat a l'air trié, il est faux.

2.3 La conversion qui échoue, et la conversion qui plie

Théorie : Theorie_08.md, section 5.

```
SELECT ('123.45')::numeric; -- fonctionne
SELECT ('n/a')::numeric;   -- ERREUR
```

Une seule valeur invalide dans un lot fait donc tomber toute la requête. PostgreSQL n'a pas de TRY_CAST, mais il a mieux depuis la version 16, et c'est la norme SQL/JSON qui le lui donne. Fabriquez trois documents pour le voir, un valide, un inconvertible, un sans la clé :

```
WITH essai(d) AS (VALUES ('{"montant": "123.45"}'::jsonb),
                          ('{"montant": "n/a"}'::jsonb),
                          ('{"autre": 1}'::jsonb))

SELECT d ->> 'montant'
      JSON_VALUE(d, '$.montant' RETURNING numeric) AS par_fleche,
      JSON_VALUE(d, '$.montant' RETURNING numeric) AS tolerant,
      JSON_VALUE(d, '$.montant' RETURNING numeric DEFAULT -1 ON ERROR) AS si_invalide,
      JSON_VALUE(d, '$.montant' RETURNING numeric DEFAULT -9 ON EMPTY) AS si_absent
FROM   essai;
```

Attendu, ligne par ligne : la première rend 123,45 dans les quatre colonnes ; la deuxième rend n/a en texte, puis un vide, puis -1, puis un vide ; la troisième rend trois vides et -9. Lisez-le bien, c'est tout l'intérêt de l'écriture. JSON_VALUE fait le chemin, la conversion et la tolérance en une expression, et il distingue deux échecs que TRY_CAST confond : la valeur était là mais inconvertible, ou le chemin ne menait à rien. Ajoutez ERROR ON ERROR et vous retrouvez le comportement strict, quand une valeur fautive doit arrêter un chargement plutôt que de le traverser.

Vérifiez maintenant que le piège de la clé nulle survit à cette écriture :

```
WITH essai(d) AS (VALUES ('{"montant": null}'::jsonb))
SELECT d ->> 'montant' IS NULL
      JSON_EXISTS(d, '$.montant') AS fleche_vide,
      JSON_VALUE(d, '$.montant' RETURNING numeric) AS cle_presente,
      JSON_VALUE(d, '$.montant' RETURNING numeric) AS converti
FROM   essai;
```

Attendu : la flèche rend un vide, la clé est pourtant là, et la conversion rend un vide elle aussi. Une requête qui compte les valeurs écartées par WHERE d ->> 'montant' IS NOT NULL passe donc à côté de ce document et le confond avec celui où la clé manque. C'est JSON_EXISTS qui les sépare, et l'exercice P8 s'appuie là-dessus.

2.4 Aplatir le tableau des lignes

Théorie : Theorie_08.md, section 6.

```
SELECT c.document ->> 'commande' AS commande,
       (ligne ->> 'produit')::int AS produit,
       (ligne ->> 'qte')::int AS qte,
       (ligne ->> 'prix')::numeric AS prix
FROM   labo08.commande AS c,
       LATERAL jsonb_array_elements(c.document -> 'lignes') AS ligne
ORDER BY commande
LIMIT 20;
```

Notez le -> avant 'lignes' : on descend, donc flèche simple. jsonb_array_elements attend un document, pas du texte.

Vérifiez le compte :

```
SELECT COUNT(*) AS lignes_aplaties
FROM   labo08.commande AS c,
       LATERAL jsonb_array_elements(c.document -> 'lignes') AS ligne;
```

```
SELECT COUNT(*) AS lignes_reelles FROM sales.salesorderdetail;
```

Attendu : les deux comptes coïncident. Puis remarquez ce que PostgreSQL ne vous donne pas : la position dans le tableau. Le json_each de DuckDB la rendait gratuitement, dans sa colonne key ; ici il faut l'ajouter avec WITH ORDINALITY.

```
SELECT c.document ->> 'commande' AS commande, ligne.position, ligne.valeur ->> 'produit' AS produit
FROM   labo08.commande AS c,
       LATERAL jsonb_array_elements(c.document -> 'lignes') WITH ORDINALITY AS ligne(valeur, position)
LIMIT 10;
```

Refaites maintenant le même aplatissage avec JSON_TABLE, la forme normalisée :

```
SELECT c.document ->> 'commande' AS commande, l.*
FROM   labo08.commande AS c,
       JSON_TABLE(c.document, '$.lignes[*]'
                  COLUMNS (position FOR ORDINALITY,
                             produit int      PATH '$.produit',
                             qte      int      PATH '$.qte',
                             prix     numeric PATH '$.prix')) AS l
```

```
ORDER BY commande
LIMIT 6;
```

Attendu : les six premières lignes de la commande 43659, numérotées de 1 à 6, avec les produits 776, 777, 778, 771, 772 et 773. Comparez les deux écritures. Les conversions sont remontées dans la déclaration des colonnes, donc on ne peut plus en oublier une ; la position vient sans WITH ORDINALITY ; et le résultat s'emploie comme une table ordinaire. Vérifiez qu'on compte toujours juste :

```
SELECT COUNT(*) AS par_json_table
FROM labo08.commande AS c,
     JSON_TABLE(c.document, '$.lignes[*]' COLUMNS (qte int PATH '$.qte')) AS l;
```

Attendu : les mêmes 121 317 lignes qu'en flèches et qu'en relationnel.

Deux comportements à connaître avant de l'adopter. Une valeur inconvertisible ne fait pas tomber la requête, elle vide sa colonne, comme le JSON_VALUE de 2.3 et avec la même clause DEFAULT ... ON ERROR par colonne. Et un tableau vide fait disparaître son document, exactement comme jsonb_array_elements : c'est LEFT JOIN LATERAL JSON_TABLE(...) ON true qui le ramène, le LEFT du Cours 07 pour la troisième fois.

```
WITH essai(nom, d) AS (VALUES ('plein', '{"lignes":[{"qte":2}]}'::jsonb),
                              ('vide',  '{"lignes":[]}'::jsonb))
SELECT e.nom, l.qte
FROM   essai AS e
LEFT   JOIN LATERAL JSON_TABLE(e.d, '$.lignes[*]' COLUMNS (qte int PATH '$.qte')) AS l ON true;
```

Attendu : deux lignes, dont « vide » avec une quantité vide. Retirez le LEFT et il n'en reste qu'une.

Partie 3 — L'index GIN, ce que l'entrepôt n'a pas

C'est la partie propre à cette voie. Le labo jumeau n'a créé aucun index : DuckDB relisait les documents à chaque requête, et sa sous-partie 3.5 l'a chiffré. Ici, on peut en demander un, et le mesurer.

3.1 Mesurer sans index

Théorie : Theorie_08.md, section 8.

```
EXPLAIN ANALYZE
SELECT COUNT(*) FROM labo08.commande WHERE document @> '{"client": 29825}';
```

Attendu : un parcours séquentiel de toute la table. Notez le temps.

3.2 Créer l'index et remesurer

Théorie : Theorie_08.md, section 8.

```
CREATE INDEX idx_commande_doc ON labo08.commande USING GIN (document);
ANALYZE labo08.commande;
```

```
EXPLAIN ANALYZE
SELECT COUNT(*) FROM labo08.commande WHERE document @> '{"client": 29825}';
```

Attendu : un parcours d'index, et un temps nettement inférieur. Vous venez de faire, en petit, ce que le Cours 10 fera en grand.

3.3 Le piège de l'index qui ne sert pas

Théorie : Theorie_08.md, section 8.

```
EXPLAIN ANALYZE
SELECT COUNT(*) FROM labo08.commande WHERE document ->> 'client' = '29825';
```

Attendu : **retour au parcours séquentiel**, malgré l'index. L'index GIN sert les opérateurs de conteneur (@>, ?, ?&) et pas les accès par flèche. C'est la leçon à retenir : un index n'aide pas parce qu'il existe, il aide parce que la requête est écrite de façon à l'employer. Le Cours 10 donnera un nom à cette propriété, la sargabilité.

3.4 Les opérateurs propres au JSON

Théorie : Theorie_08.md, sections 4 et 8.

```
-- Les documents qui possèdent la clé « remise ».
SELECT COUNT(*) FROM labo08.commande WHERE document ? 'remise';

-- Ceux qui possèdent les deux clés à la fois.
SELECT COUNT(*) FROM labo08.commande WHERE document ?& array['commande', 'lignes'];
```

Attendu : zéro pour le premier, tout pour le second. Un mot d'avertissement pratique : l'opérateur ? entre en conflit avec le marqueur de paramètre de plusieurs pilotes applicatifs, qui l'interprète comme une variable à substituer. La parade qu'on voit souvent conseillée, `jsonb_exists`, n'est pas une fonction documentée : c'est la mise en œuvre interne de l'opérateur, que le catalogue décrit comme telle, « implementation of ? operator ». La fonction documentée qui pose la même question est `jsonb_path_exists(document, '$.remise')`, et elle rend le même compte, mesuré. Les deux ont un prix commun : une fonction n'emploie pas l'index GIN, et le plan repasse au parcours séquentiel. Avant de réécrire, cherchez dans la documentation de votre pilote s'il sait échapper le ?.

Partie 4 — La comparaison qui conclut le module

Théorie : Theorie_08.md, section 9.

```
-- Depuis le document.
SELECT c.document ->> 'commande' AS commande,
       SUM((ligne ->> 'qte')::int * (ligne ->> 'prix')::numeric) AS total
FROM   labo08.commande AS c,
       LATERAL jsonb_array_elements(c.document -> 'lignes') AS ligne
GROUP BY 1
ORDER BY total DESC
LIMIT 5;

-- Depuis le relationnel.
SELECT d.salesorderid AS commande, SUM(d.orderqty * d.unitprice) AS total
FROM   sales.salesorderdetail AS d
GROUP BY 1
ORDER BY total DESC
LIMIT 5;
```

Passez EXPLAIN ANALYZE devant les deux, ou \timing on dans psql, et comparez les temps. Sur notre poste, la version document a pris environ 180 ms et la relationnelle environ 80 ms, un peu plus du double : chaque extraction est une analyse du document. L’entrepôt du labo jumeau montrait un écart d’une trentaine de fois, parce que DuckDB, qui range ses tables par colonnes, y agrège le relationnel bien plus vite, alors que le document ne profite pas de ce rangement.

La question à trancher — Laquelle des deux écritures garderiez-vous en production, et pourquoi ? Un bon raisonnement pèse trois choses : le coût de lecture pour l’humain, le coût d’exécution pour la machine, et ce qui arrive le jour où le fournisseur ajoute un champ. La réponse n’est pas la même selon que la donnée d’origine est relationnelle ou vient vraiment d’une API.

Partie 5 — Ce que le contraste vous a montré

Théorie : Theorie_08.md, section 10.

Sujet	PostgreSQL, ce labo	DuckDB, Labo_08_OLAP.md
Les types	json et jsonb, l’expérience du doublon les distingue	JSON seul, un texte validé à l’entrée
Clé de valeur nulle	conservée avec null	conservée avec null
Fabriquer l’imbrication	sous-requête corrélée, le schéma est normalisé	GROUP BY suffit, l’étoile a déjà rapproché

Sujet	PostgreSQL, ce labo	DuckDB, Labo_08_OLAP.md
Extraire	quatre opérateurs, -> et ->> à ne pas confondre, plus la famille normalisée	les mêmes flèches, entre parenthèses dans une comparaison
Conversion invalide	la requête tombe, sauf JSON_VALUE ... RETURNING, qui rend NULL	TRY_CAST rend NULL et continue
Position dans le tableau	WITH ORDINALITY, ou FOR ORDINALITY dans un JSON_TABLE	json_each la rend gratuitement, en texte
Indexer	GIN, à créer et à mesurer soi-même	aucun index, chaque requête relit les documents
L'index qui ne sert pas	démontrable : @> l'emploie, ->> non	sans objet
Charger un fichier	hors du SQL, côté applicatif ou par COPY texte	read_json_objects, et son option format pour un tableau

La leçon de fond dépasse la syntaxe. Sur le transactionnel, le semi-structuré est un corps étranger : il faut lui construire un index à la main, la conversion est intolérante, et rien n'est optimisé pour lui. L'entrepôt de la formation ne le traite pas beaucoup mieux : DuckDB le garde en texte et le relit à chaque requête. Les entrepôts infonuagiques vont plus loin : d'après sa documentation, Snowflake analyse son VARIANT à l'écriture et range à part les chemins qui reviennent. Cela ne veut pas dire qu'il faut en mettre partout dans l'entrepôt : la section 9 de la théorie reste valable des deux côtés.

Nettoyage

DROP SCHEMA labo08 **CASCADE**;

Et ensuite ?

Vous avez vu les deux terrains sur le même parcours. Le LATERAL sert ici une troisième fois, après la sous-requête du Cours 07 et avant l'as-of join du Cours 09, où il ira chercher « la valeur en vigueur au moment de l'événement ». Trois usages, une seule idée.

Prochaine étape : les exercices, dans Exercices_08.md.